

CAT WASHROOM READING AUTOMATION — COMPLETE REFERENCE

Every detail needed to replicate this automation from scratch. Last updated: April 9, 2026.

1. OVERVIEW

When the cat washroom light (light.cat_lights) turns ON, the app:

1. Checks server startup cooldown (30s) — ignores events during first 30s after boot
2. Determines light state from webhook body (state/action fields) or queries HA API (up to 3 retries with 1s delays, plus a verification query 500ms later if first says ON)
3. If light OFF → stops all playback, saves chunk progress, sends stop to tablet/TV, clears session
4. If light ON → checks cooldowns, then plays "One moment, checking your readings" via HA Cloud TTS on HA Voice speaker
5. Gets active semester settings from DB (storage.getActiveSemesterSettings)
6. Calculates current week number using `getWeekNumber(today, semesterStartDate, readingWeekStart)`
7. Finds next unlistened file by course priority (`findNextFileByPriority`)
8. If no cached files → syncs from OneDrive, then retries
9. Asks "Good morning/afternoon Bryn. Would you like to play [file description]?" via HA Cloud TTS
10. Waits up to 23 seconds for confirmation (webhook or input_boolean poll every 10s)
11. If no confirmation → stores late-confirm window (10 min), plays CHUM FM 104.5
12. If confirmed → plays "Okay, I will now play [file description]" via HA Cloud TTS on HA Voice speaker
13. Extracts PDF text, chunks it (~1400 chars each), generates Edge TTS audio per chunk
14. Plays chunks sequentially on Nest speaker, tracking progress in DB
15. On completion of all chunks → marks file as listened, auto-advances to next file

2. ALL HOME ASSISTANT ENTITIES

- **light.cat_lights** — Light trigger entity (ON starts automation, OFF stops it)
- **media_player.home_assistant_voice_097c38_media_player** — HA Voice speaker in cat washroom (used for TTS prompts and confirmations via tts/speak)
- **media_player.bathroom_speaker** — Nest speaker (used for chunk playback via play_media, and as TTS fallback)
- **media_player.echo_cat_left_am** — Echo Left in cat washroom
- **media_player.echo_cat_right_am** — Echo Right in cat washroom
- **media_player.echo_cat_washroom_middle** — Echo Middle in cat washroom
- **media_player.cat_washroom_media_group** — Cat WR Media Group (3 echos + nest combined)
- **media_player.tv_cat_wr** — Samsung TV in cat washroom
- **media_player.fire_tv_172_24_0_88** — Fire Stick (ADB control for Silk browser on TV)
- **media_player.tablet_cat** — Tablet in cat washroom (shows PDF reader page)
- **tts.home_assistant_cloud** — HA Cloud TTS engine entity
- **media_player.byhome** — Everywhere speaker group (all speakers in house)
- **input_boolean.module_reading_pending** — HA boolean: set ON when prompt is active
- **input_boolean.module_reading_confirmed** — HA boolean: set ON by HA automation when user confirms

3. HOME ASSISTANT CONNECTION

- **URL:** <https://ec8ebfanqrqlsnmnggrdl4yzq2i8koah.ui.nabu.casa> (Nabu Casa cloud URL)
 - **Token:** Stored in env var HOME_ASSISTANT_TOKEN — long-lived access token starting with eyJ...
 - **Override:** HOME_ASSISTANT_URL_OVERRIDE env var can override the URL
 - **All HA API calls** use Authorization: Bearer {token} header
-

4. DEPLOYED APP URL

- **Env var:** DEPLOYED_APP_URL — on Pi this is `http://172.24.1.204:5000`
 - **Used for:** Serving TTS audio files to speakers (e.g. `http://172.24.1.204:5000/tts-audio/file.mp3`)
 - **Also used for:** PDF reader URL sent to tablet and TV for visual follow-along
-

5. ALL WEBHOOK ENDPOINTS

All are POST requests from HA automations to the app at `http://172.24.1.204:5000`:

- **POST /api/webhook/cat-lights** — Main trigger. Body can include: `{ state: "on"/"off" }, { new_state: { state: "on" } }, { action: "turn_on"/"turn_off" },` or `{ to_state: "on"/"off" }`. If body doesn't contain state, queries HA API directly (3 retries).
 - **POST /api/webhook/cat-lights-confirm** — Voice/button confirmation. Resolves the pending confirmation promise. Also handles late confirmations within 10-min window (stops CHUM FM, starts reading). On confirm, stops media on cat washroom Echos (unless Spotify Everywhere group is active).
 - **POST /api/webhook/cat-shower-button** — Physical shower button press. Skips the prompt entirely, immediately finds next file and starts playback. If playback already active or prompt pending, ignores (button is just toggling shower/fan).
 - **POST /api/webhook/cat-wash-stop** — Toothbrush/knob stop trigger. Stops all cat washroom playback.
 - **POST /api/webhook/cat-knob-press** — Knob rotation. Body: `{ action: "volume_up" | "volume_down" | "stop" }`
 - **POST /api/webhook/voice-command** — Voice commands. Body: `{ command: string }`. Supported commands: "pause", "resume"/"rezoom"/"re-zoom", "stop", "skip", "back", "repeat", "status", "clear_player"
-

6. HA AUTOMATIONS REQUIRED IN HOME ASSISTANT

1. **Cat Lights State Change:** Trigger on `light.cat_lights` state change → POST to `http://172.24.1.204:5000/api/webhook/cat-lights` with `{ state: trigger.to_state.state }`

2. **Cat Shower Button:** Trigger on physical button entity → POST to `http://172.24.1.204:5000/api/webhook/cat-shower-button`
 3. **Confirmation Boolean:** When `input_boolean.module_reading_confirmed` turns ON → POST to `http://172.24.1.204:5000/api/webhook/cat-lights-confirm`
 4. **Voice Command:** Via HA assist pipeline or custom intent → POST to `http://172.24.1.204:5000/api/webhook/voice-command` with `{ command: "..."}`
 5. **No-Motion Auto-Stop:** HA automation `"automation.motion_cat_wr_no_motion"` — triggers after N minutes of no motion. The app reads this automation's duration to announce the pause timeout.
-

7. FULL LIGHT-ON FLOW (STEP BY STEP)

1. **Webhook received:** POST `/api/webhook/cat-lights`
2. **Server startup cooldown:** If server started less than 30s ago (`SERVER_STARTUP_COOLDOWN_MS = 30000`), return "ignored"
3. **State resolution:** Parse body for `state/action/to_state`. If none found, wait 1s then query HA API for `light.cat_lights` state (up to 3 attempts, 1s between). If first query says ON, verify with second query 500ms later (catches brief on/off flickers).
4. **Light OFF handling:** `stopNestPlaybackWithGoodbye('light_off')`, stop TTS session, `stopAllCatWashroomSpeakers`, send `stop_playback` to `tablet+TV`, clear playback session, set `lastPlaybackStoppedAt`
5. **Light ON — Cooldown checks:**
 - If `catLightsPromptPending` is true → skip (prompt already in progress)
 - If `lastPlaybackStoppedAt` was less than 60s ago → skip (post-stop cooldown)
 - If `catWashManuallyStoppedAt` was less than 120s ago → skip (manual-stop cooldown)
 - If playback already active and not stale → skip. Stale = chunk still at 0 after 3 min, or started more than 10 min ago
6. **Verify light still ON:** Query HA API — if light is actually OFF, abort
7. **Set prompt pending:** `catLightsPromptPending = true`, `catLightsLastPromptAt = now`
8. **Return 200 immediately:** `{ action: "processing" }` — rest runs in background

9. **HA restart window check:** If 4:00-4:10 AM Toronto time, play "Home Assistant is currently restarting..." TTS, wait until 4:10, then re-check light
10. **Early device wake (parallel):** Wake tablet via ADB (KEYCODE_WAKEUP + brightness 255)
11. **Immediate prompt (parallel):**
 - Turn on HA Voice + Nest speakers (media_player/turn_on)
 - Set HA Voice volume to 0.50, Nest volume to 0.35
 - Turn off input_boolean.module_reading_confirmed, turn on input_boolean.module_reading_pending
 - Wait 1500ms
 - Play "One moment, checking your readings." via tts/speak on HA Voice (primary). Fallback: Edge TTS on Nest speaker
 - Wait 3500ms after ack plays
12. **Get semester settings:** storage.getActiveSemesterSettings() — if null, abort
13. **Calculate week number:** getWeekNumber(torontoDate(), semesterStartDate, readingWeekStart)
14. **Find next file:** findNextFileByPriority(allFiles, weekNumber)
 - If no cached files → syncOneDriveFilesForWeek, then retry
 - If cached file has priority > 1 → sync OneDrive first to check for higher-priority files
 - If cached file is priority 1 → use it, sync OneDrive in background
15. **Wait for prompt + device wake to finish**
16. **Re-check light still ON** — abort if off
17. **No files found:** Play CHUM FM 104.5 on cat washroom media group, clear prompt pending
18. **File found — build TTS prompt:** greeting (Good morning/afternoon based on hour) + "Would you like to play [file description]?"
19. **Play prompt:** tts/speak on HA Voice speaker (primary). Fallback: Edge TTS on Nest. If all fail: play CHUM FM

20. **Wait 2s, then check light still ON**

21. **Wait for confirmation:** Up to 23 seconds. Two methods:

- Primary: catLightsConfirmResolve callback (set by /api/webhook/cat-lights-confirm)
- Backup: Poll input_boolean.module_reading_confirmed every 10s

22. **Reset booleans:** Turn off both module_reading_pending and module_reading_confirmed

23. **Not confirmed:** Store late-confirm file + week + 10-min expiry. Play CHUM FM radio.

24. **Confirmed:** Check light still ON one more time. If off, abort.

25. **Start playback:** confirmTTS = "Okay, I will now play [file description]." → startConfirmedPlaybackFlow()

8. startConfirmedPlaybackFlow() — DETAILED

1. Pause audio preparation (audioPreparationPaused = true)
2. Calculate resumeFromChunk: $\max(0, \text{savedChunk} > 0 ? \text{savedChunk} - 1 : 0)$ — replays 1 chunk of context
3. If resuming, remove checkmark from context chunk in DB
4. Build PDF reader URLs for tablet (master) and TV (followOnly)
5. Increment catWashSessionId, set catWashPlaybackActive = true
6. Start toothbrush polling (checks if toothbrush starts → stops playback)
7. Send tablet navigation command (tablet-nav polling)
8. Start text extraction from PDF (async)
9. Set HA Voice speaker volume to 0.45
10. Await text extraction
11. Chunk text into ~1400 char pieces
12. Pre-generate first chunk TTS audio (parallel with confirmation)
13. **PARALLEL BLOCK (Promise.allSettled):**

- Tablet setup (navigate to reader URL)
- TV setup (turn on Samsung TV via WoL, wake Fire Stick, launch Silk browser, navigate to followOnly URL, set immersive mode)
- First chunk audio pre-generation
- **Confirmation TTS:** Set HA Voice volume to 0.50, play "Okay, I will now play..." via tts/speak on HA Voice speaker (primary). Fallback: tts/speak on Nest speaker. Wait confirmWaitMs = $\max(4000, (\text{wordCount}/160)*60000 + 2000)$ milliseconds.

14. After all settle: Set Nest volume to 0.75 for playback

15. Start startNestChunkPlayback()

9. TTS MESSAGE DELIVERY (CRITICAL FIX — Apr 9 2026)

Problem: Confirmation and pause messages were using Edge TTS audio generation → upload to HA → play_media on Nest speaker. This pipeline was unreliable — messages cut off mid-word and sometimes didn't play at all.

Fix (commit f7c88c20c): Changed both confirmation and pause TTS to use HA Cloud TTS (tts/speak) directly on the HA Voice speaker — the same method used for the initial prompt messages which work reliably.

- **Confirmation ("Okay, I will now play..."):** Now uses tts/speak → HA Voice speaker (primary), with tts/speak → Nest speaker as fallback
- **Pause ("Paused. Say re-zoom to continue..."):** Now uses tts/speak → HA Voice speaker (primary), with tts/speak → Nest speaker as fallback
- **Chunk playback:** Still uses Edge TTS → uploadAudioToHA → play_media on Nest speaker (this works for long chunks)
- **All prompt messages ("One moment...", "Would you like to play..."):** Already used tts/speak on HA Voice (unchanged)

10. CHUNK PLAYBACK (startNestChunkPlayback)

1. If no chunks (text extraction failed), skip file, try next unlistened file
2. Load pre-prepared audio cache from file.preparedAudioPaths

3. Load checked chunks set from file.checkedChunks — skip already-checked chunks
 4. For each chunk (starting from resumeFromChunk):
 - Check abort/session match — if aborted, save progress and break
 - Skip if chunk is in checkedChunksSet
 - Every 3 chunks: play attention prompt "Bryn, are you paying attention?"
 - Get audio: pre-prepared cache → pre-generated first chunk → look-ahead → generate on-the-fly
 - Estimate duration: $\max(5000, (\text{wordCount}/175) * 60000 + 1000)$ ms
 - Play on Nest speaker via playOnNestSpeaker(). If 3 consecutive failures: check HA connectivity, wait up to 5 min for reconnection
 - Start look-ahead generation for next chunk (parallel)
 - Wait estimated duration
 - Save lastChunkIndex to DB
 5. After all chunks: mark file as listened, auto-advance to next unlistened file
-

11. playOnNestSpeaker() — HOW AUDIO PLAYS

1. media_player/media_stop on Nest (clear previous)
 2. media_player/volume_set 0.75 on Nest
 3. If audio path starts with /tts-audio/ or /prepared-audio/: uploadAudioToHA() — reads file from dist/public/, uploads as multipart form to HA media source, gets Nabu Casa URL
 4. If HA upload succeeded: play_media on Nest with Nabu Casa URL, wait 3s, check state. Trust it worked (no Cloud TTS fallback)
 5. If HA upload failed and ttsText available: fallback to nestCloudTTSPay() — tts/speak on Nest via HA Cloud TTS
 6. If no ttsText: use direct URL from app server (<http://172.24.1.204:5000/tts-audio/...>)
-

12. uploadAudioToHA() — AUDIO UPLOAD PIPELINE

1. Read audio file from disk at `dist/public/{path}`
 2. Reject if file is less than 100 bytes (corrupted)
 3. Build multipart form with boundary, `media_content_id = "media-source://media_source/local/."`, file as `audio/mpeg`
 4. POST to `{haUrl}/api/media_source/local_source/upload`
 5. Return Nabu Casa URL:
`https://ec8ebfanqrqlsnmnggrdl4yzq2i8koah.ui.nabu.casa/media/local/{fileName}`
-

13. VOICE COMMANDS (POST `/api/webhook/voice-command`)

- **"pause"**: Aborts chunk loop, stops Nest speaker, saves progress, sets `voiceCommandPauseState` with auto-stop timer (10 min default). Plays "Paused. Say re-zoom to continue, or I'll stop in N minutes." via `tts/speak` on HA Voice speaker. N = fetched from HA automation duration (`automation.motion_cat_wr_no_motion`).
 - **"resume" / "rezoom" / "re-zoom"**: Restores from `voiceCommandPauseState`. Re-fetches file from DB, re-extracts text, re-chunks, resumes from saved `chunkIndex`. Plays "Resuming [file description]."
 - **"stop"**: Full stop. Stops all speakers, clears pause state, turns off Fire Stick + TV. Plays "Stopped. Your reading position has been saved."
 - **"skip"**: Marks current file as listened (sets `listened=true` in DB). Finds next file. If found: starts playback with "Skipped. Now playing [next file]." If not: plays CHUM FM.
 - **"back"**: Goes back one chunk (`max(0, currentChunk - 1)`). Restarts chunk playback from there.
 - **"repeat"**: Replays current chunk.
 - **"status"**: Announces "Currently playing [file], chunk [N] of [total]."
 - **"clear_player"**: Nuclear reset — clears all playback state, stops all speakers, clears pause state.
-

14. LIGHTS OFF BEHAVIOR

1. If catWashPlaybackActive: call stopNestPlaybackWithGoodbye('light_off') — which aborts chunk loop, stops Nest + HA Voice, saves chunk progress, plays goodbye TTS, turns off Fire Stick + Samsung TV, sends tablet stop
 2. If TTS session active: stopTTSSession()
 3. stopAllCatWashroomSpeakers: stops Nest, all 3 Echos, media group (both stop + pause commands). If Spotify Everywhere group active: only stops Nest + media group, preserves Echo playback
 4. Send stop_playback command to tablet (master) and TV
 5. If no playback was active: skip Fire Stick/Samsung TV shutdown (avoids CEC waking devices)
 6. Clear catLightsPromptPending, catWashPlaybackTrigger, playback session
 7. Set lastPlaybackStoppedAt if playback was actually active
-

15. SHOWER BUTTON (POST /api/webhook/cat-shower-button)

1. Server startup cooldown check (30s)
 2. If playback already active → skip (button is toggling shower/fan)
 3. If prompt already pending → skip
 4. Get semester settings, calculate week number
 5. Find next file by priority (same logic as lights)
 6. If no files → play CHUM FM
 7. No prompt — immediately start playback: "Okay, I will now play [file description]."
 8. catWashPlaybackTrigger = 'button'
 9. startConfirmedPlaybackFlow()
-

16. TIMING AND COOLDOWNS

- **SERVER_STARTUP_COOLDOWN_MS:** 30,000ms (30s) — ignores all light events during first 30s after boot

- **CAT_LIGHTS_PROMPT_COOLDOWN_MS:** 180,000ms (3 min) — minimum time between prompts
- **Post-stop cooldown:** 60,000ms (60s) — after playback stops, ignore light-on events for 1 min
- **Manual-stop cooldown:** 120,000ms (120s) — after manual stop command, ignore for 2 min
- **Stale playback detection:** chunk at 0 after 3 min, OR started more than 10 min ago
- **TTS ack wait:** 3,500ms after "One moment..." plays
- **Pre-wake wait:** 1,500ms after turning on speakers before playing ack
- **Confirmation timeout:** 23,000ms (23s) — max wait for user confirmation
- **Late confirmation window:** 600,000ms (10 min) — can confirm after CHUM FM starts
- **Confirmation wait:** $\max(4000, (\text{wordCount}/160)*60000 + 2000)$ ms after confirmation TTS
- **Chunk duration estimate:** $\max(5000, (\text{wordCount}/175)*60000 + 1000)$ ms per chunk
- **Attention interval:** Every 3 chunks, play "Bryn, are you paying attention?"
- **Edge TTS timeout:** 10,000ms per chunk generation attempt
- **No-motion auto-stop:** Default 10 min (read from HA automation duration). Stops paused playback.
- **HA restart window:** 4:00-4:10 AM Toronto — waits for HA to finish restarting
- **Toothbrush polling:** Checks every few seconds if toothbrush entity starts running → stops playback

17. COURSE PRIORITY

Lower number = higher priority. Determines which course plays first:

- **Winter 2026:** CPPA122=1, CFNF400=2, CASL101=3
- **Spring/Summer 2026:** CECN210=1, CPHL110=2, CHIS105=3

- **All CPPA variants (101,102,120,121,122,124,125):** priority 1

Stored in DEFAULT_COURSE_PRIORITY constant. Also loads from persistent-uploads/course-play-priority.json (defaults always override disk values). Spring/summer half detection: isInSpringSummerHalfA() checks date ranges.

Fallback priorities: Professional Required courses = 100, Liberal Studies = 200, Open Electives = 300, Unknown = 999

18. FILE SELECTION LOGIC (findNextFileByPriority)

1. Filter all files to current week number (folder matches week-{N})
2. Exclude files with extraction failures (extractionFailedFileIds set)
3. Filter to unlistened only
4. If ANY unlistened MODULE files exist → only consider modules (readings blocked until all modules done for that week)
5. For each eligible file, compute: coursePriority (from getCoursePriorityForFile), weekNum, isModule
6. Sort: coursePriority ASC, weekNum DESC, modules before readings
7. Return top result

19. TTS COURSE NAMES (spoken aloud)

- CPPA122 → "Local Politics"
- CFNF400 → "Human Sexuality"
- CASL101 → "Sign Language"
- CECN210 → "Economics"
- CPHL110 → "Philosophy"
- CHIS105 → "History"

describeFileForTTS(): Returns "your {courseName} {Module|Reading} for week {N}". Detects course code from folder or filename. Detects module vs reading from folder/filename.

20. ONEDRIVE FILE SYNC (syncOneDriveFilesForWeek)

OneDrive folder structure: /School/1.

TMU/Courses/{year}/{semester}/{courseCode}/{Module|Reading}/Week {N}/

1. Get courses from semester settings via getSemesterOneDriveCourses()
2. For each course: list week folders, find matching week number
3. For each sub-type (module, reading): list PDF files
4. For each PDF not already in DB: download, save to persistent-uploads/, create DB record
5. Queue new files for audio preparation (prepareFileAudio)

Audio preparation: Background process that pre-generates Edge TTS for all chunks and stores paths in file.preparedAudioPaths

21. CHUM FM RADIO FALLBACK

When no files available or confirmation not received: plays CHUM FM 104.5 on cat washroom media group.

Command: media_player/play_media with entity_id:

media_player.cat_washroom_media_group, media_content_type: "custom",
media_content_id: "play 104.5 chum fm"

Blocked if catWashPlaybackActive is true.

22. SPOTIFY EVERYWHERE GROUP PROTECTION

If Spotify is playing on the Everywhere group (media_player.byhome), the automation protects the cat washroom Echo speakers — only stops Nest + media group, preserves Echo Spotify playback. Checked via isSpotifyPlayingOnEverywhere() which reads spotifyActivePlaybacks set.

23. TV SETUP (inside startConfirmedPlaybackFlow)

1. Turn on Samsung TV via WoL (media_player/turn_on for media_player.tv_cat_wr)

2. Check Fire Stick state via HA API
 3. If Fire Stick off/standby: turn on directly (CEC wakes Samsung TV)
 4. If Fire Stick already on: OFF→ON cycle for CEC wake
 5. Send KEYCODE_WAKEUP via ADB
 6. Launch Silk browser via monkey command or am start
 7. Navigate to followOnly PDF reader URL
 8. Set immersive mode (hides status bar)
-

24. PLAYBACK STATE PERSISTENCE

Playback session stored in app_state table with key "playback_session":

- fileId, fileName, chunkIndex, totalChunks, trigger, startedAt, updatedAt, status (active/paused)
 - Saved on: playback start, each chunk advance, pause, stop
 - Cleared on: completion, full stop
 - Used for: resume after server restart, status display in dashboard
-

25. DATABASE REQUIREMENTS

Tables used by automation:

- **semester_settings:** semesterStartDate, readingWeekStart, course1/2/3 codes, course1/2/3_display_name (TEXT — see CRITICAL FIX below)
 - **files:** id, folder, original_name, display_name, listened (boolean), last_chunk_index (integer), checked_chunks (text/JSON array), total_chunks (integer), prepared_audio_paths (text/JSON array), object_path
 - **app_state:** key-value store. Key "playback_session" stores active/paused session JSON. Key "raw_story_top3" stores Raw Story monitor state.
-

26. CRITICAL FIX — MISSING DATABASE COLUMNS (Apr 9 2026)

Problem: The automation broke because the semester_settings table on the Pi's database was missing the course1_display_name, course2_display_name, and course3_display_name columns. These columns were added to the Drizzle schema but never pushed to the Pi's database via migration.

Root cause: The code calls storage.getActiveSemesterSettings() which SELECTs all columns including display_name columns. When these columns don't exist in the database, the query fails silently or returns an error, causing the entire automation to abort at step 11 (getting semester settings).

Fix: Manually run these ALTER TABLE commands on the Pi's PostgreSQL database:

```
ALTER TABLE semester_settings ADD COLUMN IF NOT EXISTS course1_display_name text;
```

```
ALTER TABLE semester_settings ADD COLUMN IF NOT EXISTS course2_display_name text;
```

```
ALTER TABLE semester_settings ADD COLUMN IF NOT EXISTS course3_display_name text;
```

Prevention: After any schema change in shared/schema.ts, always verify the Pi database has the new columns. The Pi uses a separate PostgreSQL instance that doesn't auto-migrate.

27. ENVIRONMENT VARIABLES

- **HOME_ASSISTANT_TOKEN** — Long-lived HA access token (starts with eyJ...)
- **HOME_ASSISTANT_URL_OVERRIDE** — HA URL (optional, defaults to Nabu Casa URL above)
- **DEPLOYED_APP_URL** — App URL for TTS audio serving (http://172.24.1.204:5000 on Pi)
- **DATABASE_URL** — PostgreSQL connection string
- **SITE_PASSWORD** — Auth password for PDF reader URLs

28. PI DEPLOYMENT

Deploy command:

```
cd ~/Home-View && git pull origin main && npm run build && fuser -k 5000/tcp; sleep 2 && pm2 delete dashboard; pm2 start dist/index.cjs --name dashboard --cwd
```

```
/home/byhomeyyz/Home-View --node-args="-r /home/byhomeyyz/Home-View/preload.cjs" && pm2 save
```

- **PM2 process name:** dashboard
 - **preload.cjs:** Required for environment variable loading at runtime
 - **Working directory:** /home/byhomeyyz/Home-View
 - **Port:** 5000
 - **Git push command:** git push
https://Reachushere:\$GITHUB_PERSONAL_ACCESS_TOKEN3@github.com/Reachushere/Home-View.git main
-

29. COMMIT HISTORY

- **2741793c1** — Working baseline (Apr 9, 2026) — automation restored after display_name column fix
 - **f7c88c20c** — Fix confirmation and pause TTS: use HA Cloud TTS on HA Voice speaker instead of Edge TTS + Nest pipeline
-

30. SERVER STARTUP BEHAVIOR

On boot, the server:

- Resets input_boolean.module_reading_confirmed and input_boolean.module_reading_pending to OFF
- Starts 30-second cooldown timer (SERVER_STARTUP_COOLDOWN_MS)
- Loads course priority from disk (persistent-uploads/course-play-priority.json), with DEFAULT_COURSE_PRIORITY always overriding
- Checks for persisted playback session in app_state — can resume after crash